

NUMPY(SHORT NOTES)

BATCH :-8

NIRANJAN SAH

YEAR:-2025-26

Library:- A library in Python is a collection of pre-written code that you can use so you don't have to write everything from scratch.

Module:- A module is just a single Python file (ending in .py) that contains code. If a Library is the entire "toolbox," a Module is a "single tool" inside that box.

NumPy:- this library is use for basic mathematical calculation.

❖ How to install NumPy?

- Open CMD as administrator and type "pip install numpy" and hit enter and wait for installation.

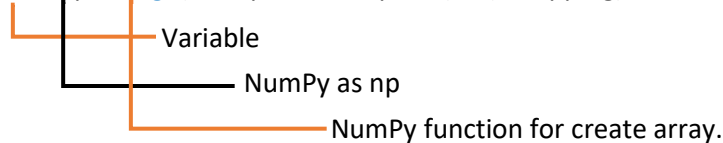
❖ How to import NumPy in jupyter notebook?

- Open Jupyter notebook and type import numpy as np.

Array:- it is like list, but homogeneous data. Fixed size, multidimensional data structure and ordered.

➤ How to create a array?

Arr= np.arange(start point, end point(n-1), stepping)



Ex:- Arr=np.arange(0,10) (without stepping, python by default take stepping +1)

print(Arr)

Result:- array([1, 2, 3, 4, 5, 6, 7, 8, 9])

Ex:- Arr=np.arange(1,11,2) (with stepping +2)

Print(Arr)

Result:- array([1, 3, 5, 7, 9])

type(Arr) #To check type of Arr

Result:- numpy.ndarray

Zeros:- it will make array with only Zero(0) value.

Ex:- arr=np.zeros(10) How may value do you need of zero

Print(arr)

Result:- array([0., 0., 0., 0., 0., 0., 0., 0., 0., 0.])

ones:- Same as zeros but value will be 1.

Ex:- arr=np.ones(10)

Print(arr)

Result:- array([1., 1., 1., 1., 1., 1., 1., 1., 1., 1.])

➤ Convert list into array.

l=[1,2,3,4]

arr=np.array(l)

print(arr)#(try its your self)

Ex:-arr=np.array([1,2,3,4])

Ex:- l=[[1,2,3],[4,5,6],[7,8,9]]

arr=np.array(l)

print(arr) #(try its your self)

Linspace:- linspace (short for "linear space") is a function used to create a sequence of numbers that are **evenly spaced** over a specific interval. Basically, you tell Python where to start, where to end, and how many numbers you want in between. Python then does the math to make sure the "**gap**" between each number is exactly the same.

NUMPY(SHORT NOTES)

BATCH :-8

NIRANJAN SAH

YEAR:-2025-26

Ex:-arr=np.linspace(start point,end point, number of value)

Arr=np.linspace(1,20,10)

Print(Arr)#try its your self

eye:-to make diagonal matrix

Ex:-arr=np.eye(4) # to make square matrix(4 by 4)

Print(arr)

➤ **Random of NumPy**

This is module of NumPy no need to import.

1. Ex:- arr=np.random.rand(2)

NumPy module

Function for generate random no. between 0 and 1 (by default)

Number of values

Print(arr)

2. Ex:- arr=np.random.randn(5)

Function same as rand but it will generate random value near to 0 and 1 with -ve and +ve values

Number of values

3. Ex:- arr=np.random.randn(5,5) # it will generate 5 by 5 matrix with random value near to 0 & 1 including -Ve & +Ve value (you can change it 2 by 4 or etc). you can do same in rand

Print(arr)

4. Ex:- arr=np.random.randint(1,100)

Function, same as rand but it will generate random value only int.

Random value between 1 and 100(you can change it)

5. Ex:- arr=np.random.randint(1,100,4)

How many values you need between 1 and 100

Note:-in EX 4 you will get only one value(random value) between 1 and 100 if you not mention how many value you need.

6. Ex:- arr=np.random.seed(4)

To lock the random value on the basic of Key

Key:- random value will not change if you don't change the key

arr=np.random.rand(5)

You will have to use both fun. in together for get result

➤ **reshape**:- function to make any array into matrix

Ex:- arr=np.arange(1,26)

arr.reshape(5,5)

Row

Column

➤ **shape**:- Function to check how many row and column present in matrix.

Ex:- arr=np.arange(1,26)

arr1=arr.reshape(5,5)

arr1.shape

Result (5,5)

➤ **max**:- function to get max value from list or array.

Ex:-arr=np.arange(0,24)

arr1=arr.max()

Result:-23

➤ **min**:- function to get min value from list or array.

Ex:- arr=np.arange(0,24)

arr1=arr.min()

NUMPY(SHORT NOTES)

BATCH :-8

NIRANJAN SAH

YEAR:-2025-26

Result:-0

- **argmax**:- Function to get the value which is present on max index.

Ex:- `arr=np.arange(0,24)`

`arr1=arr.argmax()`

Result:-23

- **argmin**:- Function to get the value which is present on min index.

Ex:- `arr=np.arange(0,24)`

`arr1=arr.argmin()`

Result:-0

- **Index in array**

Ex:- `arr=np.arange(26,50)`

`arr[0]`

Result:- 26 — Zero index or 1st value

- **Slicing in array**

Ex:- `arr=np.arange(26,50)`

`arr[0:2]`

— Zero index or 1st value
— 2nd index or (n-1) index
— To

- **With stepping**

Ex:- `arr=np.arange(26,50)`

`arr[0:6:2]`

— Stepping (by default +1)

- ❖ How to get a particular value from matrix using index.

Ex:- `arr = np.arange(1,25)`

`arr1=arr.reshape(3,8)`

`arr[1][2]`

— Row index
— Column index

```
array([[ 1,  2,  3,  4,  5,  6,  7,  8],
       [ 9, 10, 11, 12, 13, 14, 15, 16],
       [17, 18, 19, 20, 21, 22, 23, 24]])
```

Broadcasting:- In NumPy, **broadcasting** is a powerful feature that allows you to perform mathematical operations between arrays of **different shapes**.

Normally, in math, you can only add or multiply two matrices if they are the exact same size.

Broadcasting allows Python to "stretch" the smaller array so that it matches the shape of the larger one, making the math possible.

Let's try first for list

Ex:- `l=[1,2,3,4]`

`l+2` (we want to add 2 with each value of list) this is possible in array not in list.

Result:- `TypeError`

Ex:- `arr=np.arange(11,20)`

`arr+10`

Notes:- You can do same for all mathematical operator with array (try).

NUMPY(SHORT NOTES)

BATCH :-8

NIRANJAN SAH

YEAR:-2025-26

❖ Is array mutable?

Yes.

1.Ex:- `arr=np.arange(11,20)`

`Arr[0]=99 # assign value 99 on 0th index`

Result:- `array([99, 12, 13, 14, 15, 16, 17, 18, 19])`

2. Ex:- `Slice_arr=arr[0:5]`

`Print(Slice_arr)`

Result:- `array([99, 12, 13, 14, 15])`

3. Ex:- `Slice_arr[:]=99`

`Print(Slice_arr)`

Result:- `array([99, 99, 99, 99, 99])`

Now lets print a original array

`Print(arr)`

Result:- `array([99, 99, 99, 99, 99, 16, 17, 18, 19])`

Did you notice! `arr` and `Slice_arr` is deferent array but still they connected to each other this is the another feature of **broadcasting**.

➤ Copy:- Function to make a copy of original array.

Ex:-`arr=np.arange(1,10)`

`arr2=arr.copy()`

2D Array

Ex:- `l=[[1,2,3],[4,5,6],[7,8,9]]`

`arr=np.array(l)`

`print(arr)`

```
array([[1, 2, 3],
       [4, 5, 6],
       [7, 8, 9]])
```

index in 2D array

1.Ex:- `arr[1][1]`

Result:-5

2.Ex:-`arr[1]`

Result:-`[4,5,6]`

Slicing in 2D array

1.Ex:- `arr[0:2,1:]`

For row(zero index to 2nd index)

For column(1st index to last index)

➤ Conditional array:-

Ex:- `arr=np.arange(0,10)`

`Print(arr)`

`arr>4`

Result:-`[0 1 2 3 4 5 6 7 8 9]`

`array([False, False, False, False, False, True, True, True, True, True])`

Ex:-`bool_arr=arr>4`

OR `arr[arr>4]`

`arr[bool_arr]` #it will get the value which is grater than 4

➤ Array with array function:-

Ex:-`arr1=np.arange(91,101)`

`Arr2=np.arange(11,21)`

NUMPY(SHORT NOTES)

BATCH :-8

NIRANJAN SAH

YEAR:-2025-26

Run:-arr1+arr2

Notes:- You can do same with other mathematical operator but value should be same in both array.

➤ Universal array function:-

- Square root

Ex:-arr=np.arange(0,10)

np.sqrt(arr) #Square roots calculation

- Trigonometric:-

Ex:-np.sin(arr) # For sin value

np.cos(arr) #For cos value

np.tan(arr) #For tan value

- Exponential value:-

Ex:- np.exp(arr)

- Log:-

Ex:-np.log(arr) #For log value

➤ Summary function:-

Ex:-arr=np.arange(0,10)

arr.sum() #For calculate sum of all value of array.

arr.mean() #For calculate arithmetic average of array.

arr.min()

arr.max()

➤ Axis logic:-

Ex:- arr=np.array([[1,2,3],[4,5,6],[7,8,9]])

Arr.sum(axis=0) #axis 0 means work on Row

Arr.sum(axis=1) #axis 1 means work on Column